

GÉNIE LOGICIEL

GeoHealth Documentation



Date: 2025-2026

AUTHORS: ANAS DAMLAKHI, JULIEN JAMOULLE, ILIAS
ESSARHIRI, HOURIA ARIOUAT, THOMAS BUSONI, LOUIS BOMAL

1 Preface

1.1 Background Information

This document is the **technical documentation** for the GeoHealth software, developed as part of the INFOM114 course at the University of Namur.

The project was carried out for over four months by a team of seven Software Engineering students, following the Scrum methodology.

Whether you are a project user or a curious reader seeking to explore the technical aspects of the project in greater depth, this documentation is for you. By the end of this guide, you will have a clear understanding of the project's source code, architecture, database, and all the necessary elements you need to fully grasp the software and extend it with new functionalities.

Considerable efforts were invested in the development of this project to produce a robust and well-designed application. However, as this work was carried out in an academic context by students, some imperfections or errors may still be present. We nevertheless hope that you will enjoy exploring this project as much as we enjoyed developing it.

1.2 How is this documentation organized

This technical documentation is split into different sections, each with its own purpose. Here is a summary of what is contained in each section.

- **Preface:** This section contains information about the context in which the project was done
- **Introduction:** This section contains a brief explanation of what this project is and what its purpose is. As it is important to understand at least the main goal of the codebase before jumping into it
- **Project Setup:** This section explains all the things you need to know to launch the project for the first time and be able to see the website in action
- **Architecture:** This section gives you a general idea of how the codebase is organized, but also explains why we chose the technological stack used in this project instead of another. This is really interesting to read if you want a more general point of view of the project (to keep the big picture in mind)
- **Codebase Overview:** This section jumps into the codebase in more detail and gives an overview of the file structure of the project. It also explains in a more precise way the purpose of particular files or packages. This section is intended to be used before working on a specific part of the codebase, to quickly understand it and highlight any important specificities you should be aware of.

- [Adding new code](#): This section makes a recap of all the steps you should think of and do if you want to add new code to the codebase. This can be great to have a clear idea of what you need to do in the end, as there is a lot of information in all the different sections
- [CI/CD](#): This section explains all the CI/CD processes used during this project: the technologies used, the pipeline workflow, etc.

1.3 Acknowledgements

The spelling of this document was reviewed with the help of [Reverso](#), [ChatGPT](#), and [Claude](#) under our supervision, as English is not our native language.

Contents

1	Preface	1
1.1	Background Information	1
1.2	How is this documentation organized	1
1.3	Acknowledgements	2
2	Introduction	6
2.1	A short remainder of what the project is	6
2.2	What is the added value of this project	6
3	Project Setup	7
3.1	Install the prerequisite environment	7
3.2	How to launch the software	7
3.2.1	Cloning the GitHub Repository	8
3.2.2	Opening the project and creating the env file	8
3.2.3	Installing Docker Database	9
3.2.4	Launching the backend	9
3.2.5	Launching the frontend	12
4	Architecture	14
4.1	System Overview	14
4.1.1	Technology Justification	14
4.1.2	System Communication Overview	14
4.2	Backend Architecture	15
4.3	Frontend Architecture	16
4.3.1	Frontend framework	16
4.3.2	Use of TypeScript	17
4.3.3	Geospatial Visualization with Leaflet	17
4.3.4	Interactive Editing with Geoman.io	18
4.4	Database Architecture	18
4.4.1	What we use for the database and why	18
4.4.2	Several points of view of the database	18
4.5	Communication Between Layers	20
4.5.1	Data Formats Between Layers	20

4.5.2	Request Lifecycle Example	21
5	Codebase Overview	23
5.1	Project Structure	23
5.2	Backend	23
5.2.1	converter	24
5.2.2	webgis	25
5.2.2.1	admin	25
5.2.2.2	annotations	26
5.2.2.3	evaluationform	27
5.2.2.4	map	28
5.2.2.5	measure	29
5.2.2.6	report	31
5.2.2.7	security	32
5.2.2.8	user	33
5.3	Frontend	34
5.3.1	core	36
5.3.1.1	rest-api-management	36
5.3.1.2	service	37
5.3.2	features	38
5.3.3	shared	39
5.3.3.1	components	39
5.3.3.2	models	40
5.3.4	Transloco	40
6	Adding new code	41
6.1	Before adding the code	41
6.2	Adding code to the backend	41
6.3	Adding code to the frontend	42
7	CI/CD	44
7.1	Pipeline overview	44
7.2	Continuous Integration	44
7.2.1	Automated Testing	44

7.2.2	Code Quality Analysis	44
7.3	Continuous Deployment	44
7.3.1	Overview	44
7.3.2	Deployment Pipeline	45
7.4	Deployment Server	45
7.4.1	Software installed	45
7.4.2	VM File Structure	46
7.4.3	Nginx configuration	46
7.4.4	SSL certification	47
7.4.5	GitHub Required Secrets	47
7.4.6	Useful Commands	47

2 Introduction

Even if this documentation serves a technical purpose. It seems necessary to at least give you the main point of this project before you start jumping into it, as it is sometimes easier to understand a project (even the technical aspects) when we understand why this project exist in a first time.

2.1 A short remainder of what the project is

Since COVID-19, which caused an almost total paralysis of health and socio-economic systems worldwide, it has become clear that pandemics represent a major challenge for modern societies. However, COVID-19 is not the only pandemic to threaten our world. Numerous infectious diseases(e.g., Ebola and Rift Valley fever) continue to emerge and re-emerge across different regions, particularly in tropical regions where ecological and environmental conditions facilitate their spread.

To anticipate these threats, it is crucial to have reliable risk maps validated and regularly updated by experts in the field. and to show them to decision-makers. This is especially important to guide interventions and improve the response of health authorities.

However, here is the problem: only a few accessible tools exist to visualize risk maps correctly, and when they exist, they are mainly designed for well-trained users. As a result, decision-makers and other non-expert stakeholders may encounter difficulties in accessing, interpreting, and using these maps efficiently.

It's in this context that we were asked to develop a webGIS platform accessible from a standard web browser, which will allow users to access to up-to-date risk maps easily and without requiring any specific abilities. The platform also aims to promote collaboration between experts by allowing expert users to provide structured feedback on the different risk maps through evaluation forms. Of course, the project includes many additional features beyond those described here but with that you have enough information to go further in the documentation and discover them by yourself.

2.2 What is the added value of this project

The project:

- Ease the access to geographical data to political decision-makers and national experts by providing an accessible interface without needing them to download specialized software.
- Promote collaboration between professionals through shared and synchronized data.

3 Project Setup

In this section, you will find everything you need to start from scratch and run the project for the first time.

3.1 Install the prerequisite environment

Before starting the project, the first step is of install the required environment. Since each software component has its own installation guide, you will be redirected to the official documentation for each tool. However, particular attention must be paid to installing the specific versions indicated here to avoid version compatibility issues.

Software	Group Version	Recommended Version	Documentation
Java	22.0.2	Stable versions between 17 and 22	Link to docs
R with the irr package (Not required to launch)	4.5.3	Latest stable	Link to docs
GDAL (Not required to launch)	GDAL 3.12.2 "Chicoutimi"	Latest stable	Link to docs
Maven	3.9.9	3.9.x	Link to docs
Angular	Core: 21.1.2	21.1.x	Link to docs
	Node.js: 22.14.0	Latest compatible	
	npm: 10.9.2	Latest compatible	
Git	2.46.2	Latest stable	Link to docs
Docker Desktop	/	Latest stable	Link to docs
IntelliJ IDEA	2025.3.2	Latest stable	Link to docs
GitHub Desktop (Not required to launch)	/	Latest stable	Link to docs

It's required to install all the software listed in the table in the given order, otherwise you may encounter issues (for example, installing Maven before Java will cause problems).

All the "not required to launch" software are as they indicated not required. However, omitting them will disable specific features:

- **R:** Without R, the metrics computation feature will be partially unavailable. Specifically, the Krippendorff metric relies on the `RCaller` library, which requires R to be installed and accessible.
- **GDAL:** Without GDAL, all features related to raster map processing will be unavailable, as GDAL is required to convert raster data into tiles.
- **GitHub Desktop:** This tool does not affect any application feature. However, we recommend installing it as it significantly simplifies Git repository management.

This documentation will assume that you are already familiar with all these tools. If you are not familiar with one of them, go take a look on Internet, there is plenty tutorial and exercises there.

3.2 How to launch the software

If you are here, it means that you have installed all the required environment to start the project (If this is not the case please refer to [Install the prerequisite environment](#) section). Now you can try to launch the project for the first time.

3.2.1 Cloning the GitHub Repository

The first step to launch the project is to obtain the project source code on your computer. The only way to do this is to clone the project GitHub Repository. You can do it by opening a terminal and by typing the following command.

```
git clone https://github.com/jjamoull/GeoHealth.git
```

Warning: The copy will be created in the current path of the terminal, so be careful to place it somewhere you can easily find it.

3.2.2 Opening the project and creating the env file

Once the repository has been cloned, open the "GeoHealth" folder you have created in IntelliJ IDEA. Make sure that the root of the project is the "GeoHealth" folder.

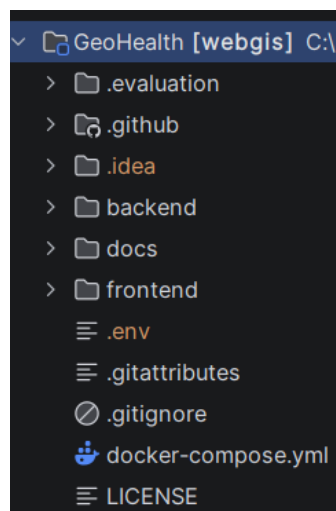


Figure 1: Project file structure

Your project structure should be similar to the one shown

Now you need to create the ".env" file. To do so you can simply add a new file at the root of the project and call it ".env" and copy the following content.

```
1 DB_USER= test
2 DB_PASSWORD= test
3 JWT_SECRET= test
4 SUPERADMIN_SECRET= test
```

Figure 2: .env content

3.2.3 Installing Docker Database

When the env is created, you can launch Docker, open a terminal and IntelliJ and type the following command

```
docker compose up -d
```

If it works, you should now have a container named "geohealth" running on Docker. This is the container that contains the database of the project (PostgreSQL).

3.2.4 Launching the backend

Now that Docker is running, the next step is to launch the SpringBoot backend

Warning: Keep in mind that when you want to launch the backend, the "geohealth" Docker container must always be running on your computer.

To launch the backend, the easiest way is to use IntelliJ IDEA and create a run configuration, as it will automatically link SpringBoot with the ".env" file created earlier.

The first thing you need to do to create a run configuration is to verify that IntelliJ IDEA detects the "backend" folder as a Maven project. If so, a small blue square will appear in the bottom right corner of the folder.

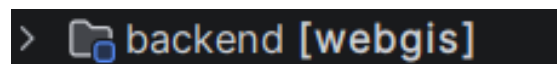


Figure 3: Backend recognized as maven project

If this is already the case, you can proceed to the next step. If not, you should indicate this to IntelliJ by yourself. To do this, simply open the 'backend' folder, right-click on the "pom.xml" file, and select "Add as Maven Project".

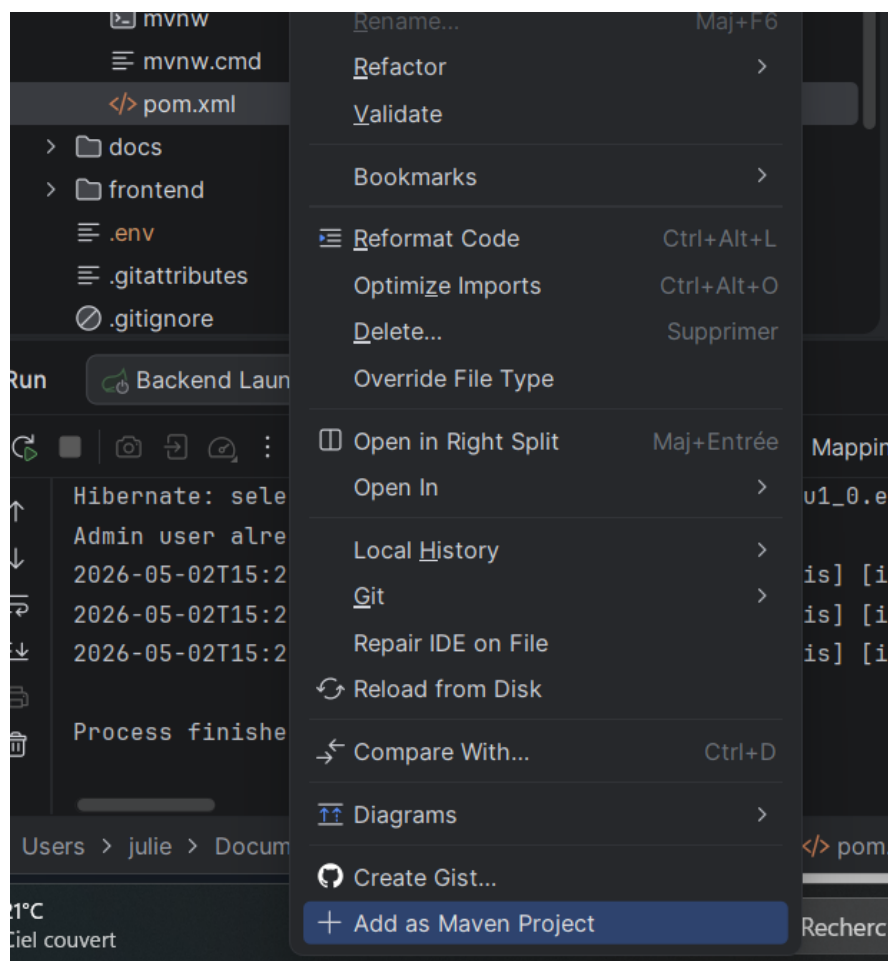


Figure 4: Add a maven project

Once IntelliJ recognises your project as a Maven one, you can create the run configuration. To do this, look at the top of the IntelliJ interface. You will see a slider labelled 'Current File'. Click on it. A drop-down menu will open. On this menu, click the option labelled 'Edit configurations...'.

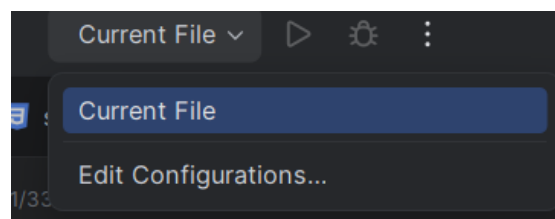


Figure 5: Edit configurations

If you click on it and if you do not have any configurations, the following screen will appear. Click on the blue link labelled 'Add new run configuration...' and look for SpringBoot.

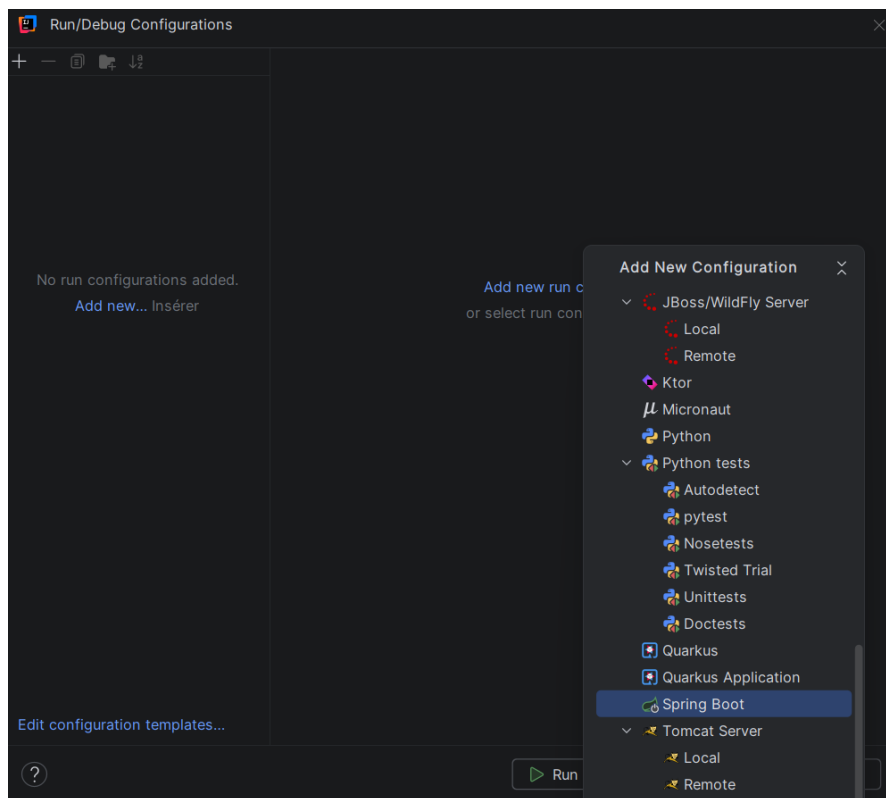


Figure 6: Adding springboot configuration

The configuration window will open. Once you are on it, the only remaining task is to activate the environment variables. To do so, simply click on the blue "Modify options" link and activate "Environment variables" by clicking on it.

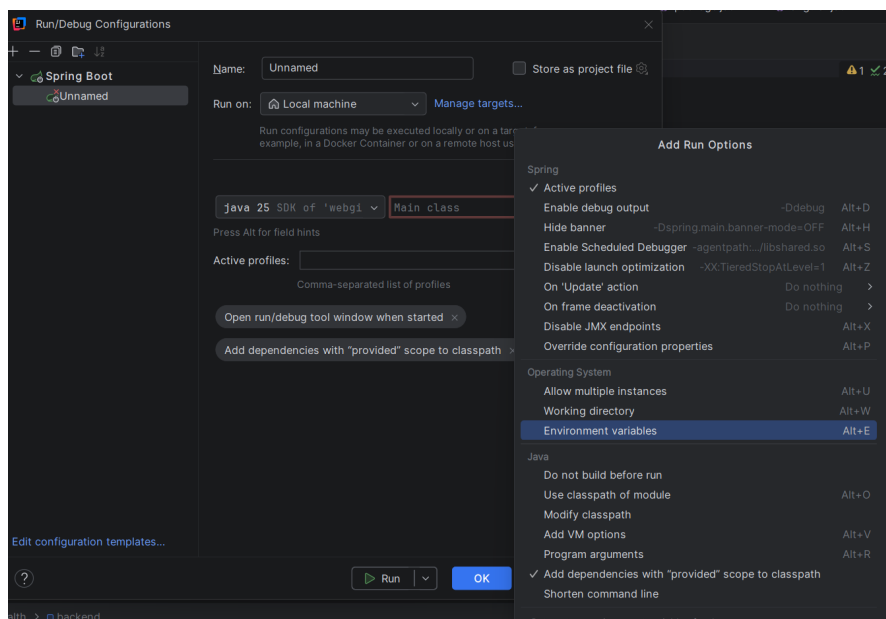


Figure 7: Adding env

Then you can simply enter all the same content that in the screenshot below and click on 'OK'.

Warning: Of course, for the ".env" file path, you should use the one on your computer, not the one in the screenshot.

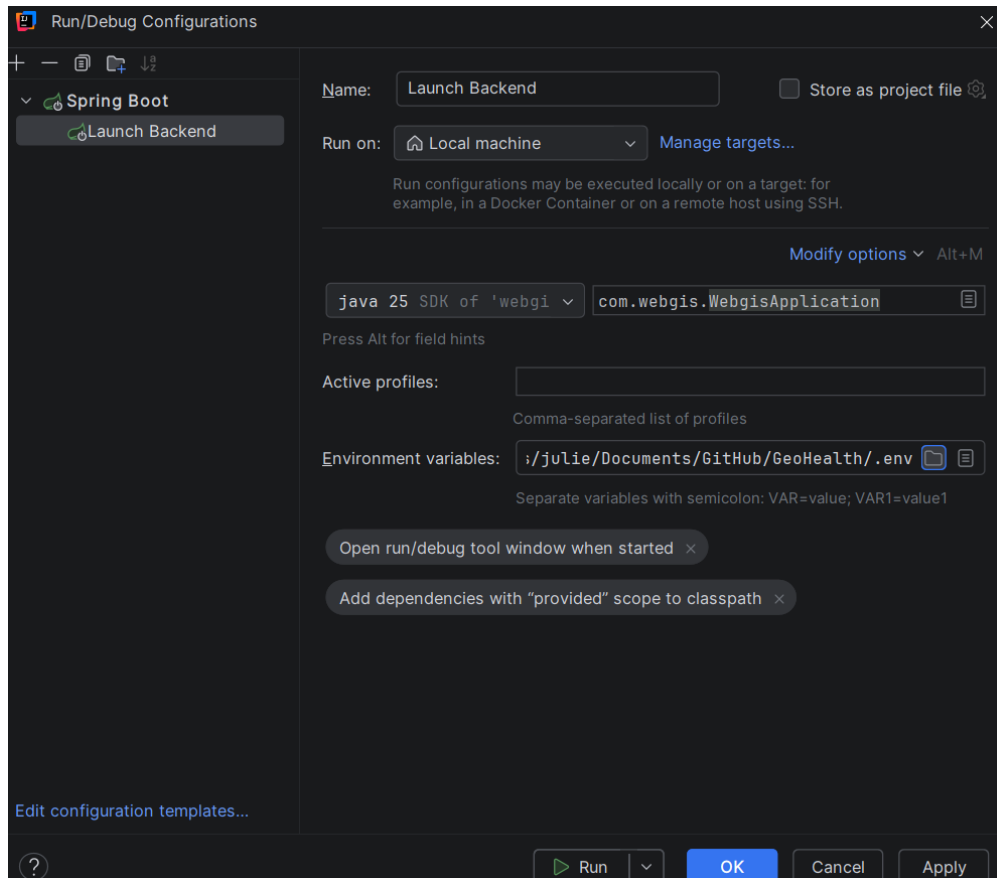


Figure 8: Configuration with content

Now if you want to launch the backend, you can click on the launch button on the right of your configuration and see the result.

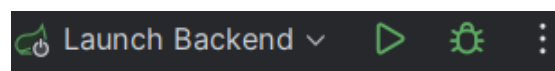


Figure 9: Button to launch the backend

3.2.5 Launching the frontend

Now that the backend and the Docker database are running, the next and final step is to launch the frontend. Here you won't need to create a configuration don't worry. The only thing you need to do is to navigate in the frontend folder (of course don't close the terminal where your backend is running) and you can type the following command.

```
npm install
```

 (Install all the dependencies. Do it only the first time)

```
ng serve
```

 (Launch the server)

Once the server is running, you can open <http://localhost:4200/> and see the website in action.

4 Architecture

In this section, you will have all the information required to understand the GeoHealth software from a global perspective, which means the architecture of the backend, the architecture of the frontend, communication between the different elements, etc.

4.1 System Overview

GeoHealth is a full-stack WebGIS application. It uses Java with the SpringBoot framework for the backend, PostgreSQL as the database running inside Docker, and Angular for the frontend. The table below summarises the main components of the stack:

Layer	Technology	Role
Backend	Java / Spring Boot	REST API, business logic
Database	PostgreSQL	Data storage
Frontend	TypeScript / Angular	User interface
CI/CD	GitHub Actions	Automated build, test and integration

4.1.1 Technology Justification

- **Java / SpringBoot:** Java is a well-known language that our group is very familiar with, especially after the good practices learned during the first year of the Master's program. This was the main reason why Java and SpringBoot were chosen over other considered alternatives such as Node.js and Python's FastAPI.
- **PostgreSQL:** PostgreSQL is the database most used with Spring Boot. Furthermore, it offers an extension called PostGIS, which integrates with the project's geography data management.
- **TypeScript / Angular:** TypeScript was chosen over JavaScript for its typing, which reduces errors and facilitates the team's code comprehension. Angular was selected as the frontend framework due to its component-based architecture, which is well suited for building structured and maintainable single-page applications.
- **GitHub Actions:** GitHub Actions was chosen as the CI/CD solution as it is already integrated in the repository, requiring no third-party services. It allows the team to automate building, testing, and code style checks via Checkstyle on every push and pull request.

4.1.2 System Communication Overview

The system communication is described in the schema below. The port used by each component is indicated in parentheses:



4.2 Backend Architecture

The backend uses different layers where each layer has a distinct responsibility. Security is handled by Spring Security, which intercepts requests before they reach the application layers. The application is a classic stateless REST API, meaning no session is stored on the server, and every request must carry a valid JWT token. The layers are as follows:

- **Security Configuration:** Defines the security configuration and rules the webgis follows, it specifies which routes are public and which require authentication. Furthermore it configures the CORS to allow requests from the frontend and registers the JWT filter before Spring's default authentication filter.
- **JWT Authentication Filter:** Intercepts every request, extracts and validates the JWT token, and sets the authentication in the Spring Security context. It relies on a [CookieService](#) for cookie extraction and a [JwtService](#) for token validation.
- **Controller:** Receives HTTP requests and extracts the necessary data using DTOs (Data Transfer Objects) which are then passed to the service layer.
- **Service:** Contains the business logic it is used as an orchestrator between the controller and the Repository layer.
- **Repository:** Handles all database interactions using Spring boot's JPA.

requests follow this flow:



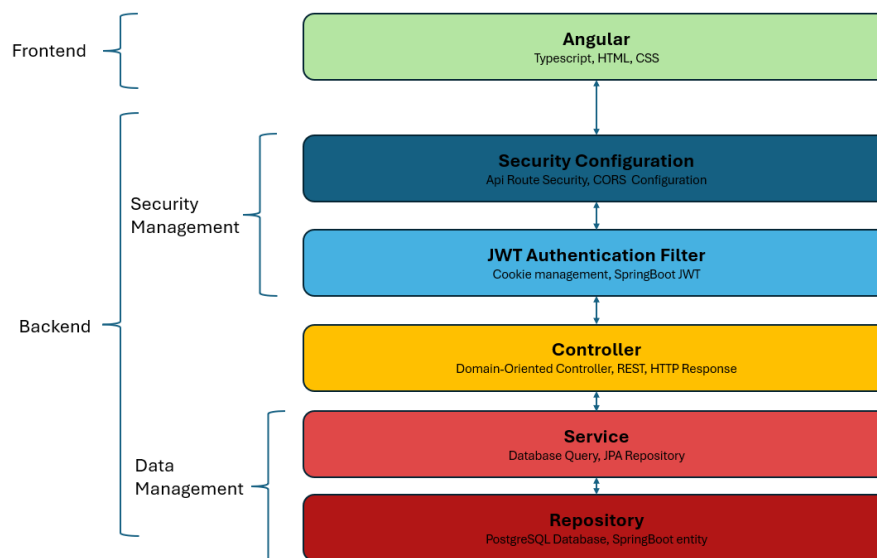


Figure 10: GeoHealth backend layers overview

4.3 Frontend Architecture

4.3.1 Frontend framework

Our frontend application is built using the Angular framework, which provides several advantages for the development of a complex, data-driven web application. Despite our effort to stick to this framework, our frontend architecture is not perfectly in line with it, which is most likely due to a lack of experience on our side regarding projects of this scale. One way our project could be improved would be to refactor it so that it aligns exactly with this architecture.

1. Unlike React, which is a UI library and Angular's first competitor, Angular is a fully integrated framework. It natively provides solutions for essential application features such as routing, HTTP communication, and access control through guards (for admin-only features). We chose Angular to reduce architectural decisions throughout the project and it helped us enhance consistency across the different parts of the codebase.
2. Another major advantage of Angular lies in the architectural structure it encourages. Angular promotes a clear separation of concerns through components, services, modules, and guards. Components are responsible for presentation logic, while services encapsulate business logic and data access. This organisation enhances maintainability, readability, and scalability of the application. The exact architecture used for this project will be explained below.
3. Angular's built-in dependency injection system simplifies component communication and promotes reusable, testable code. Services can be centrally managed and injected where needed, reducing coupling between different parts of the application.

4. The framework is particularly well suited for collaborative development. Because Angular enforces conventions and standardized project organization, potential future developers who might take over the project can quickly understand the codebase structure, reducing onboarding time and limiting architectural inconsistencies.
5. Finally, Angular integrates naturally with RESTful backend architectures such as the Spring Boot backend used in this project. Its strongly typed TypeScript environment improves reliability by detecting errors during development rather than at runtime.

4.3.2 Use of TypeScript

Angular applications are developed using TypeScript. The use of TypeScript significantly improves code reliability and maintainability by introducing strong typing, interfaces, and compile-time error detection.

In the context of this project, which manipulates structured epidemiological and geographical data, type safety is particularly important. Data objects such as maps, geographical layers, user roles, and evaluation results benefit from explicit typing, reducing runtime errors and improving developer productivity.

Additionally, TypeScript enhances team collaboration by making data structures explicit and self-documented, allowing us to better understand how information circulates throughout the application.

4.3.3 Geospatial Visualization with Leaflet

Geographical visualization represents a central component of the application. To support interactive mapping features, the project relies on the Leaflet JavaScript library, an open-source solution widely adopted for web-based geographic information systems (GIS).

Leaflet provides a lightweight and powerful framework for rendering maps, displaying GeoJSON data, and managing spatial layers. Its flexibility allows integration of multiple geographic datasets while maintaining strong performance in the browser environment.

Within the application, Leaflet is used to display risk maps, visualize administrative divisions, and support expert-driven spatial analysis. Custom Angular components encapsulate Leaflet instances, ensuring clean integration with the frontend architecture and preserving separation between visualization logic and business logic.

4.3.4 Interactive Editing with Geoman.io

To extend Leaflet’s native capabilities, the project integrates the Geoman.io plugin, which provides advanced tools for spatial editing directly within the browser.

Geoman.io enables users to draw, edit, and manipulate geographic features such as polygons and boundaries interactively. This functionality is essential for expert workflows, allowing domain specialists to annotate regions, adjust spatial divisions, and contribute directly to the analytical process.

The integration of Geoman.io transforms the application from a passive visualization platform into an interactive geospatial decision-support system. Experts can actively refine geographic information, improving data quality and supporting collaborative analysis.

Together, Leaflet and Geoman.io provide a robust foundation for interactive GIS functionalities while remaining compatible with Angular’s component-based architecture.

4.4 Database Architecture

4.4.1 What we use for the database and why

For this project, we used **Hibernate** ORM with **JPA** to create our database.

We chose these two technologies because they are widely used among SpringBoot developers and in the Java community. We also chose to use an ORM approach, as it allows us to abstract the complexity of PostgreSQL and to remain within the Java object-oriented paradigm at all times. Therefore, reducing the potential number of errors.

The only drawback of the ORM approach in our case is that it can be difficult to express more complex or specific queries. However, this was not an issue in our project, as only one query had to be written directly in SQL.

The option of writing all SQL queries manually was considered. However, it was seen as an unnecessary source of potential errors that could be avoided.

4.4.2 Several points of view of the database

Here under are several schemas that can help you gain a better understanding of the database.

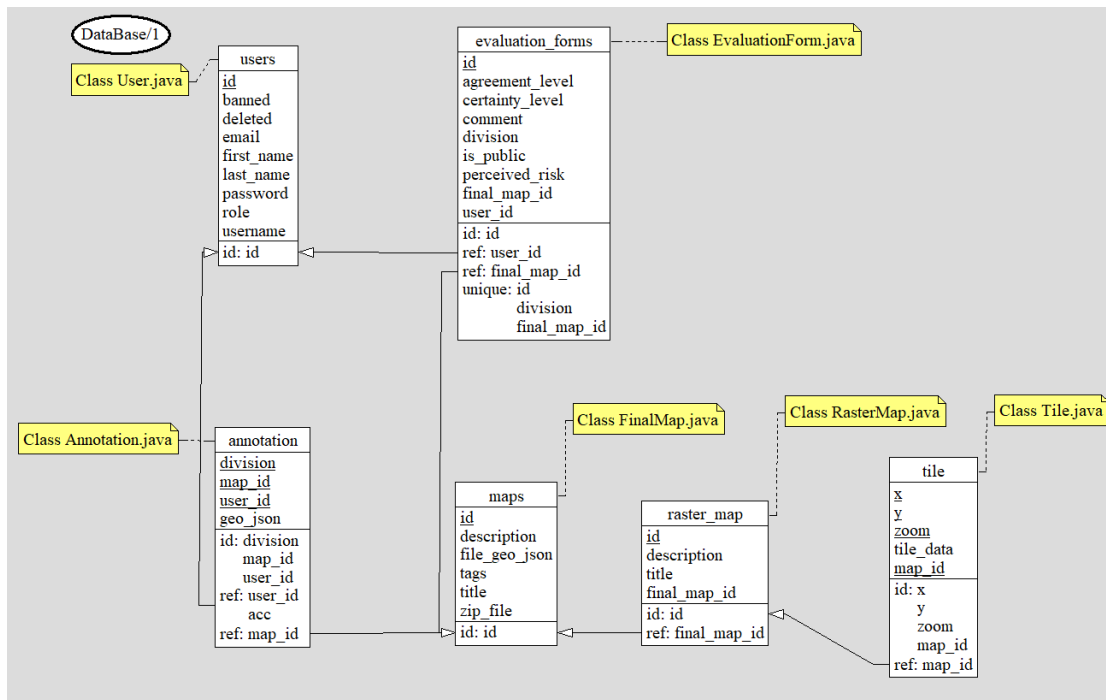


Figure 11: GeoHealth DB Schema and Java class related to each table

This schema represents what our database looks like. Note that you will never have to manipulate these tables directly, as all of them are implemented as Java classes in the program. For example, the users table corresponds to a class named User in our software. However, this schema can still help you understand how the Java classes are translated into a real database and provide a better understanding of how the main objects of our program are linked to one another (which is not always easy to understand only with the source code).

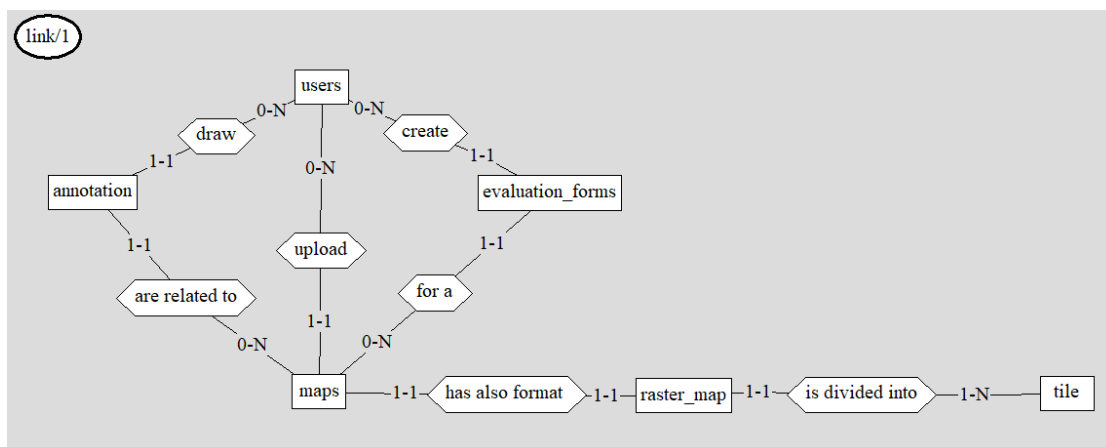


Figure 12: Link between the main objects of our program

The schema above represents illustrates the cardinalities and the main relation-

ships between the tables which are not displayed on the first one. You should see it as a very simplified resume of the systems.

Warning: This schema does not represent all existing relationships between the classes (for example, a user can perform more actions than just uploading a map). Its main purpose is to highlight the most fundamental interactions within the system, with a particular focus on cardinalities.

4.5 Communication Between Layers

GeoHealth is a WebGIS application that uses the REST API to communicate over the HTTP(S) protocol. As the server is stateless, it means that all the requests between the user and the website must be self contained meaning they must include all needed information for each request. The server needs to verify the user to identify the user, their permissions and other requests. In practice each request include the JWT token and all necessary inputs for each call.

4.5.1 Data Formats Between Layers

- Angular components are where the calls happen. These components interrogate the Angular services. The Angular component fills the models (which are DTOs) and passes them to the Angular services to perform the appropriate calls to the backend.
- The Angular service takes the DTOs provided by the component and calls the controller. For this call, it uses JSON over HTTP(S), with the JWT token (stored as a cookie) automatically sent by the browser, along with the necessary DTOs if needed.
- In the backend, once a controller endpoint is called, the request must first pass through the security configuration (except for authentication calls). It then goes through a filter that performs the necessary checks, such as verifying whether a user is banned. Finally, the request reaches the controller.
- The controller extracts the necessary data from the incoming request, including DTOs if present, and passes it to the service layer. DTOs are used here to ensure that only the relevant and expected data is received from the client, avoiding exposing the internal entity structure to the outside world. Once the service returns the data, the controller maps it into DTOs for the same reason before serializing them into JSON and sending them as the HTTP response.
- The service layer works with JPA Entity objects when communicating with the repository. Entities represent the actual database model and are managed by JPA. The service receives raw data from the controller and is responsible for invoking calls from the database. once the controller gets the necessary data from the service it will then maps them into DTOs.

4.5.2 Request Lifecycle Example

Authentication Flow The login procedure followed by a user (does not require a JWT token, as it is used to authenticate the user and issue one):

1. The user fills in their credentials on the login page. The Angular component maps them into a DTO and passes it to the Angular authentication service. The request passes through the CORS validation.
2. The Angular service sends a POST request with the credentials as JSON in the request body over HTTP(S). The request skips the JWT filter entirely.
3. The controller receives the request, extracts the credentials from the DTO and passes them to the service.
4. The service verifies the credentials against the database through the repository. If the credentials are valid, it returns the user if correct.
5. The controller generates a JWT token, attaches it as a cookie in the HTTP response header and returns a success message.
6. Angular receives the response and the browser automatically stores the JWT cookie. The user is now authenticated and all new requests will automatically have the JWT cookie.

Authenticated Request Flow The path of a connected user visiting their profile:

1. The user navigates to their profile page. The Angular profile component calls the Angular user service to fetch the current user's data.
2. The Angular service sends a GET request to `/users/profile` over HTTP(S). The browser automatically sends the JWT cookie with the call. The request passes through the CORS validation.
3. In the backend, the request first passes through the security configuration which identifies this as a protected route. It then goes through the JWT filter, which extracts the token from the cookie, validates it, and sets the authentication in the Spring Security context.
4. The request reaches the controller, which extracts the username from the JWT token using the `CookieService` and `JwtService` and passes it to the service layer.
5. The service searches for the information for the given username. The repository retrieves it from the PostgreSQL database via JPA and returns the user to the service, which then returns it to the controller.
6. The controller maps the entity into a `Dto` and returns it as JSON in the HTTP response with an ok status code.

7. Angular receives the JSON response, maps it into the corresponding TypeScript model, and the profile component displays the user's data.

5 Codebase Overview

5.1 Project Structure

This project is a website with a separate backend and frontend. As shown in the structure below, the repository is divided into the following parts:

```
GeoHealth/
|-.github/
|-.backend/
|-.frontend/
|-.docker-compose.yml
|-.env
```

- the **.github** folder which contains the GitHub Actions workflows for the CI/CD pipeline
- the **Backend** which contains the Java Spring Boot code
- the **Frontend** which contains the Angular and TypeScript code
- the **docker-compose.yml** which configures the PostgreSQL database container
- The **.env** file centralizes all the configuration and sensitive values used across the project, such as database credentials, JWT secrets, and other environment-specific settings.

5.2 Backend

The backend is split into several main folders/files:

```
backend/
|-.src/
|  |-.main/
|  |  |-.java/
|  |  |  |-.com/
|  |  |  |  |-.converter/
|  |  |  |  |-.webgis/
|  |  |  |  |  |-.admin/
|  |  |  |  |  |-.annotations/
|  |  |  |  |  |-.evaluationform/
|  |  |  |  |  |-.exception/
|  |  |  |  |  |-.map/
|  |  |  |  |  |-.measure/
|  |  |  |  |  |-.report/
|  |  |  |  |  |-.security/
|  |  |  |  |  |-.user/
|  |  |  |  |  |-.MessageDto
|  |  |  |  |  |-.WebgisApplication
|  |  |  |  |-.resources/
|  |  |  |  |-.application.properties
|  |  |-.test/
|  |-.pom.xml
|  |-.checkstyle.xml
```

- The **converter** package which is a module responsible for transforming file formats, like converting shapefiles to GeoJSON.
- The **webgis** package contains all the business logic of the application and is itself organized into the following sub-packages:
 - **admin** which handles administration features
 - **annotations** containing all the annotation logic
 - **evaluationform** containing all the evaluation logic
 - **exception** handling application exceptions
 - **map** everything related to maps
 - **measure** containing all the measurement logic
 - **report** handling report generation
 - **security** all the authentication, authorization and CORS logic
 - **user** containing all the user features.

Each of these sub packages follows the same structure (except security and exception).

- The **resources** folder contains the `application.properties` file, which is the main Spring Boot configuration file.
- The **pom.xml** file has all the java dependencies and build configuration,
- The **checkstyle.xml** file defines the code style rules enforced.
- The **WebgisApplication** is the entry point of the Spring Boot application, it is considered as a `main`.

5.2.1 converter

The **converter** package gathers all matters regarding file conversion from one format to another.

The package is organised as follows :

```
converter/  
  |- DetectFiles  
  |- ShapeFileToGeoJsonFile  
  |- TiffFiles  
  |- ZipFiles
```

- **DetectFiles**: gathers services for other class usage. It contains the methods to detect and find shapefiles in the uploaded content. It also contains a method to validate a zip file.

- **ShapeFileToGeoJsonFile**: allows for Shapefile conversion to GeoJSON format. (This GeoJSON is later used for display in the frontend and is needed for OpenStreetMap to work properly)
- **TiffFiles**: allows for tiff (.tif) files to be converted to tiles using external processes (gdal transformation)
- **ZipFiles**: allows for zip file decompression. Performs some security checks (zipbombs, zipslip attacks) before any treatment.

In order to better comprehend what the package stand for, some important matters should be addressed :

- The expected uploaded content for zip files is a compressed shapefile (set of .shp, .cpg files and other relevant extensions). Zipping was added to handles the problem of heavy files.
- Tiff files can't be properly displayed in the frontend technology we use. That is why we convert them into tiles which allows for fluid and precise display on the map. Tiles are smaller png files gathering a small part of the initial map at a certain zoom level. Tile conversion implies heavier storage for greater zoom level.

5.2.2 webgis

All sub-packages except **security** have the same logic structure, they are all responsible for a specific feature of the application. Each package contains all the needed classes for that feature: the entity, the repository, the service, the controller and the DTOs.

5.2.2.1 admin

The **admin** package is responsible for managing the features that are only accessible by the administrator of the platform (the users who have the admin role). This package is separated from the user package as the controllers in the admin package have special protection in the **SecurityConfig** and also to avoid mixing logics that are not related to one another.

Here is the package organisation:

```
admin/  
  |- dto/  
  |- evaluationform/  
    |- AdminEvaluationFromController  
  |- map/  
    |- AdminFinalMapController  
    |- AdminRasterMapController  
  |- user/  
    |- AdminController  
  |- AdminInitializer
```

- **dto**: contains all the Data Transfer Objects used for the different admin features.
- **evaluationform**: contains the controller for all the routes linked to evaluation form management by the admin.
- **map**: contains the controllers for all the routes linked to map management by the admin (such as uploading/deleting final maps and uploading raster maps).
- **user**: contains the controller for all the routes linked to user management by the admin (such as banning/unbanning a user, getting a list of all the users, or changing a user's role).
- **AdminInitializer**: the class that initializes the super admin when launching the SpringBoot application for the first time.

The code of this package is fairly easy to read and well specified. The only detail you should know before going into it are the following:

- All the backend routes starting with `/admin/**` can only be executed by a user with admin role, or they will be forbidden by the [SecurityConfig](#).
- The password of the super admin is defined in the `.env` file as `SUPERADMIN_SECRET`. Make sure to use a strong password before deploying the website, as losing access to this account would mean losing control over all other accounts.

5.2.2.2 annotations

The **annotations** package is responsible for handling all the considerations regarding annotations.

The package is organised as follows :

```
annotations/  
  |- dto/  
  |- Annotation  
  |- AnnotationController  
  |- AnnotationId  
  |- AnnotationRepository  
  |- AnnotationService
```

- **dto**: contains all the Data Transfer Object used to send data to frontend
- **Annotation**: defines the Sprint Boot entity for annotations and maps it to the database
- **AnnotationController**: defines the routes associated with annotations, allows the frontend to request for annotations and save them in the database.

- **AnnotationId**: defines the way to identify annotations in the database
- **AnnotationRepository**: handles the SQL queries to the annotation table in the database.
- **AnnotationService**: provides all the needed services for the relevant controllers to work

In order to better comprehend what the package stand for, some important matters should be addressed :

- An instance of annotation in the database is used as a **GeoJSON layer** in the frontend. An "annotation" is in fact the **set of annotations** a user adds for **one division**. For example, a circle, a text label and a square used for the **same division** will be stored as one annotation.
- Instances of annotations are identified by a combination of an user id, a map id and the division that is concerned. Regarding the division name, it is indirectly retrieved from the GeoJSON file derived from the uploaded shapefile.

5.2.2.3 evaluationform

The **evaluationform** package is responsible for managing all the evaluations made on the maps by the users of the system.

Here is the package organisation:

```
evaluationform/  
  |- dto/  
  |- EvaluationForm  
  |- EvaluationFormController  
  |- EvaluationFormRepository  
  |- EvaluationFormService
```

- **dto**: contains all the Data Transfer Objects used for the evaluation forms
- **EvaluationForm**: the Spring Boot entity representing an evaluation form and mapping it to the database
- **EvaluationFormController**: provides all the routes linked to evaluation form management (saving, updating, deleting and getting a form)
- **EvaluationFormRepository**: handles the SQL query with the database for the evaluation_form table and allows expressing them in Java
- **EvaluationFormService**: contains the business logic for evaluation form management and acts as a layer between the EvaluationFormRepository and controllers

The code of this package is fairly easy to read and well specified. The only details you should know before going into it are the following:

- In `EvaluationFormRepository`, the request *"findDivisionsWithValidPublicEvaluationForms"* is directly written in SQL. This is to be avoided in general and must be used only if there is no way to express your request with JPA. However, here the request was too hard to express with JPA. The values could have been computed manually (by looking at all the evaluation forms and making a list of the divisions they concern), but it would require iterating over all the evaluation forms, which is a huge loss in performance.
- In `EvaluationForm`, there is a unique constraint on `"user_id"`, `"division"`, `"final_map_id"`. This constraint represents the fact that for each map, a user can only make one evaluation for each division. This rule should be kept in mind when coding to avoid making mistakes.
- `EvaluationForm` objects are linked to `FinalMap`, not to `RasterMap`, as raster maps are only another format in which the final map can be displayed

5.2.2.4 map

The **map** package is responsible for everything related to maps in the application.

Here is the package organisation:

```
map/
|- finalmap/
|  |- dto/
|  |- FinalMap
|  |- FinalMapController
|  |- FinalMapRepository
|  |- FinalMapService
|  |- MapTag
|- raster/
|  |- dto/
|  |- RasterMap
|  |- RasterMapController
|  |- RasterMapRepository
|  |- RasterMapService
|- service/
|  |- TransformTifFiles
|- tile/
|  |- Tile
|  |- TileConstants
|  |- TileController
|  |- TileId
|  |- TileRepository
|  |- TileService
```

It is divided into four sub-packages:

- **finalmap**: handles the final maps, which are the maps that are displayed to the users and uploaded by the admin.
- **raster**: handles the raster maps, which are the raw map files (.tif), displayed to the users and uploaded by the admin.
- **service**: contains shared services used across the map package.
- **tile**: handles the map tiles, which are the individual pieces that compose a raster map.

The **raster** and **finalmap** modules follow the same architecture as the other modules, with controllers, services, and repositories. The **finalmap** module is responsible for managing shapefiles stored as GeoJSON. The **raster** module handles two types of maps: final maps in raster format, and risk factor maps. Since both share the same format (they are tif files), they are both represented as tiles through the **tile** package.

The key distinction between the two is in the database. When an admin creates a final map, a raster file (.tif) is uploaded and stored in the same way as a risk factor map. However, a **RasterMap** entity that corresponds to a final map will have its **finalMap** attribute populated a nullable foreign key linking it to its associated **FinalMap** entry. When this attribute is **null**, the raster is treated as a standalone risk factor map. This optional relationship is what differentiates the two use cases within a single unified **RasterMap** entity. And so when an admin needs to upload a risk factor map he uploads a single tif file and the foreign key will be null.

The **tile** package is responsible for storing and serving raster maps in a format suitable for web display. Since .tif files cannot be rendered directly in a browser, they are pre-processed and sliced into small .png image tiles indexed by a map identifier, a zoom level, and their **x** and **y** coordinates in the OpenStreetMap 2D space. These tiles are stored in the database and served to the frontend on demand through the **/tile** endpoint.

Beyond simple tile retrieval, the **tile** package also exposes a mean computation endpoint. Given a map identifier, a zoom level, and a geographic coordinate (latitude and longitude), the service resolves the corresponding tile and computes the mean pixel value of the block containing that coordinate. To do so, each tile is internally subdivided into a grid of fixed-size blocks. The PNG tile is first decompressed into a raw grayscale byte array, and the mean pixel value of the target block is then computed and normalised to a $[0, 1]$ range. This value is returned alongside the tile and block coordinates as a response.

5.2.2.5 measure

The **measure** package is responsible for computing all the metrics that are measured on a map based on the map data and the evaluation forms provided by the

users for this map.

Here is the package organisation:

```
measure/  
  |- dto/  
  |- measurecontroller/  
  |- measureservice/  
  |- RiskLevel
```

- **dto**: contains all the Data Transfer Objects used for the metrics
- **measurecontroller**: a folder containing all the controllers to get the different metrics that can be computed on a map
- **measureservice**: a folder containing all the services that compute the metrics for specific maps (called by the controllers)
- **RiskLevel**: this is an enumeration representing the different risk levels ("UNDEFINED", "LOW", "MEDIUM", "HIGH") for a division. This enumeration is here because it is easier to deal with enum constants during computation than strings. Furthermore, each constant is associated with a number which is useful in the computation.

As you can see, there is not only one class for the service and one for the controller as in most packages. This is because there are 10 metrics that must be computed, each with its own logic. Therefore, to avoid a god class, I split all the logic into 3 categories:

- **EvaluatorAgreementMeasure**: these are all the metrics which are related to how much evaluators agree with one another
- **MeanMeasure**: these are all the metrics which are the mean of the answers given by the evaluators for a specific map
- **ModelEvaluationMeasure**: these are all the metrics which are related to how much the evaluators agree with the map they have to evaluate

For each of these categories, the logic is always the same: one controller to give access to the data to the frontend and one service to compute the metrics for the controller (same logic as in the majority of the code).

Another thing that is interesting to notice is that there are two main types of metrics: the national metrics, which are computed for the whole map, and the divisional metrics, which are computed for a division of a map.

Here is a table showing which metric is national and which is divisional:

Metric	Scope
Krippendorff	National
National Average Entropy	National
National Consensus Score	National
National Model Field Agreement Score	National
Divisional Weighted Entropy	Divisional
Divisional Consensus Score	Divisional
Mean Agreement	Divisional
Mean Certainty	Divisional
Dominant Perceived Risk	Divisional
Weighted Divisional Level Agreement	Divisional

Table 1: Metrics and their scope

For the rest, I can only invite you to go read the documents called "Calculs des métriques.xlsx" which is stored in the same folder as this document to get all the formulas that were used to compute these metrics and to read the specification of the function that you are interested in.

There are just two last things you should know:

- Krippendorff's alpha is the only metric that is not directly computed in Java but is computed with R using the RCaller library. This is because this metric is hard to compute, and redoing all the code in Java would take an enormous amount of time and be a huge source of mistakes.
- The ModelEvaluationMeasure metrics are not available for EBOLA maps as they require a specific classification of the risk of each division into "LOW", "MEDIUM", "HIGH", which is not available for EBOLA maps

5.2.2.6 report

The report package is responsible for generating Excel reports for the maps. Here is the package organisation:

```
report/
  |- MeasureHolder
  |- ReportController
  |- ReportService.java
```

- **MeasureHolder**: this class is where all the metrics are computed (by calling the metrics services). This allows splitting the metrics computation from the report generation and eases the job of ReportService
- **ReportController**: provides the route to generate the report to the frontend and calls the ReportService to generate it
- **ReportService**: contains the business logic to generate the report using the Apache POI library

This package is pretty straightforward to understand as all the functions are specified, and it has a small number of services and only one route in the ReportController. However, there is one thing you should know:

- In the ReportService, in the function "createReportForMap", there are two checks that the map is not an Ebola map. This is because all the model evaluation metrics cannot be computed for EBOLA maps. See the metrics section to know more about that

5.2.2.7 security

The **security** package is different from the other packages as it is not responsible for a specific feature but for the authentication and authorization of the whole application.

It follows the following structure:

```
security/  
  |- CookieService  
  |- JwtAuthenticationFilter  
  |- JwtService  
  |- PasswordEncoderConfig  
  |- SecurityConfig
```

CookieService

The **CookieService** is a Spring service responsible for all cookie operations related to JWT authentication using a **COOKIE_NAME** and the **jwtService** which contains the logic of the jwt token. It contains three methods:

- **generateCookie**: creates the JWT cookie. The cookie is configured with **httpOnly** to prevent JavaScript from reading it, protecting against XSS attacks, **sameSite=Strict** to prevent the cookie from being sent in cross-site requests, protecting against CSRF attacks, and **secure** which is configurable via the **.env** file to say if the project works with http or https.
- **deleteCookie**: removes the cookie by giving a new cookie with the same name and a **maxAge 0**, which the browser will automatically delete.
- **getJwtFromCookie**: extracts the JWT token value from the cookies of an incoming HTTP request.

JwtAuthenticationFilter

The **JwtAuthenticationFilter** extends Spring's **OncePerRequestFilter**, meaning it intercepts every incoming HTTP request before it reaches the **SecurityConfig**. It is responsible for identifying who is making the request by extracting and validating the JWT token (also checks if the user is banned). It contains two methods:

- **doFilterInternal**: extracts the JWT token from the cookie and validates it. If valid, it loads the user from the database and checks if they are banned. If the user is not banned, it sets the authentication in the `SecurityContextHolder` so Spring Security knows who is making the request. If the token is missing or invalid the request is passed to the next filter unauthenticated.
- **authenticateUser**: creates a `UsernamePasswordAuthenticationToken` with the user and their role following Spring Security's `ROLE_` prefix convention and sets it in the `SecurityContextHolder`.

JwtService

The `JwtService` is responsible for all JWT token operations. It loads a secret key from the `.env` file and makes a hashed key via `PostConstruct` method to ensure the key is ready before the service is used. It contains four methods which are self explanatory.

PasswordEncoderConfig

This class is just used to encoding the password with `bcrypt`.

SecurityConfig

The `SecurityConfig` is a Spring configuration class that defines all the security rules of the application. It contains two beans (a bean is an object managed by the Spring container, meaning Spring is responsible for creating it, configuring it, and injecting it wherever it is needed in the application):

- **securityFilterChain**: configures the security rules of the application. CSRF is disabled as the application uses JWT tokens stored in `httpOnly` cookies which already protect against CSRF attacks. The session management is set to `stateless` meaning no session is stored on the server and every request must carry a valid JWT token as it is a REST API. The routes `/auth/login`, `/auth/register`, `/auth/status` and `/auth/logout` are public, it then defines what routes are allowed to what role. The `JwtAuthenticationFilter` is registered before Spring's default `UsernamePasswordAuthenticationFilter`. For the form login and HTTP basic authentication they are both disabled as the project is a REST API and authentication is handled on the frontend.
- **corsFilter**: configures the CORS policy of the application. It only allows requests from the frontend URL defined in the `.env` file, with credentials enabled so the JWT cookie is sent automatically. Only the `GET`, `POST`, `PUT` and `DELETE` HTTP methods are allowed, the `config.setMaxAge` defines the browser's cache for the cors.

5.2.2.8 user

The `user` package is responsible for managing the users of the WebGIS application, except for the administrator functionalities, which are handled in the dedicated [admin](#) package. Like most backend packages, the `user` package follows a simple structured organisation:

```
user/  
  |- dto/  
  |- AuthController  
  |- User  
  |- UserController  
  |- UserRepository  
  |- UserService
```

- **dto** contains all the Data Transfer Objects used for users management
- **AuthController** provides all the routes that users must use to authenticate themselves (e.g., login or registration).
- **User** the SpringBoot Entity representing a user and mapping the user data to the database.
- **UserController** provides all the routes linked to users that are not related to the authentication (such as updating user information, deleting an account, or retrieving the user profile).
- **UserRepository**: handles the communication with the database and allows queries to be executed automatically using Spring Data.
- **UserService**: contains the business logic for user management and acts as a layer between UserRepository and controllers

All the code of this package is specified and self-explanatory. Providing a detailed explanation here would mainly result in unnecessary repetition and would be slower than just reading the code by yourself.

The only information not directly written in the code, but discussed in the architecture section, is the dependency flow.

```
Angular Client → Security → Auth/UserController → UserService  
→ UserRepository → Database
```

This flow should be kept in mind while working and you should stick to it as much as possible to keep a clean and controlled dependency flow.

5.3 Frontend

The frontend structure is more complex than the backend structure, as the Angular framework already provides a skeleton that contains many pre-existing files. Therefore, here we will focus only on the folders that we have created and not the generic Angular ones. However, to not leave you without explanations about those files, I highly recommend that you at least read the [Angular file structure guide](#)

This will explain the purpose of all the generated files. Of course, we highly recommend learning more about Angular than that before jumping into this section, but we cannot explain all Angular here, so you will have to do it by yourself.

When you go into the frontend folder, it will look like this:

```
frontend/
  |- src/
  |- angular.json
  |- package-lock.json
  |- proxy.conf.json
  |- tsconfig.app.json
  |- tsconfig.json
  |- tsconfig.spec.json
```

All the .json files here are basically only configuration files which are autogenerated by Angular and are not very important for the understanding of the code.

The interesting code is inside the src folder.

```
src/
  |- app/
  |- assets/
  |- index.html
  |- main.server.ts
  |- main.ts
  |- server.ts
  |- styles.css
```

As you can see here, there are two main folders:

- **app**: This folder contains all the HTML, CSS, and TypeScript of the project. This is where you will work most of the time when you want to add a new feature to the frontend.
- **assets**: This folder contains all the static files of the frontend (images, translation files, etc.). These files are not code but they are necessary for the website to be functional.

As all the logic is in the app folder, we will dive deeper into it.

```
app/
  |- core/
  |- feature/
  |- shared/
  |- app.config.server.ts
  |- app.config.ts
  |- app.css
  |- app.html
  |- app.routes.ts
  |- app.spec.ts
```

```
| - app.ts
| - transloco.config.ts
| - transloco.loader.ts
```

The app folder has a more complex organization than the other folders, but I will first explain the general idea, and then we will focus on each of these folders individually.

- **core**: This folder is responsible for managing all the calls made to backend services and the backend endpoints
- **feature**: This folder contains all the main pages of the website with their HTML, CSS, and TS code.
- **shared**: This folder contains all the components that are shared among all the pages of the website

5.3.1 core

```
core/
| - rest-api-management/
| - services/
```

As you can see, the core package is divided into two main folders:

5.3.1.1 rest-api-management

This folder is responsible for managing all the backend endpoint paths that can be accessed by the frontend. It is divided into three files, each with its own purpose:

```
rest-api-management/
| - endpoint.ts
| - environment.prod.ts
| - environment.ts
```

- **endpoint.ts**: This is the file where all endpoints are defined. Each endpoint, which is basically a string representing an API path, is mapped to a key. For example, the endpoint `"/user/update"` is mapped to `"USER.UPDATE"` key.

Why do we do that? This approach allows flexibility in case the API paths change in the future. For instance, `"user/update"` might become `"user/updatev2"`. Without this mapping file, every occurrence of the string would need to be updated across the entire project. With this system, only the path mapped to the `"USER.UPDATE"` key needs to be modified, and all other parts of the project will automatically adapt, as they reference this file.

- **environment.prod.ts** and **environment.ts**: These related files contain the base URL of the API. There are two files because the base URL of the production API is different from the development one.

So, a full backend path such as:

```
'https://geohealth.unamurcs.be/api/user/update'
```

will be represented as:

```
'${this.baseUrl}${API_ENDPOINTS.USER.UPDATE}'
```

when used in services. This approach allows easy modification of both the base URL and the endpoint paths. Therefore, reducing the coupling of the entire project to the backend endpoint path.

5.3.1.2 service

The service folder, as its name suggests, is where all the backend services are called.

```
service/
  |- AdminService/
  |- AuthService/
  |- EvaluationFormService/
  |- LoginService/
  |- MapService/
  |- MeasureService/
  |- ReportService/
  |- UserService
```

As you can see, the organization of the services is very similar to the backend. This is because this folder essentially contains only backend calls written in TypeScript, allowing us to easily use the backend API in the frontend.

All the services are organized in the same way:

- A series of imports to include all dependencies: importing `HttpClient` to make HTTP requests, importing the environment and endpoint files, importing the required DTOs, etc.
- Declaration of variables: declaration of the API base URL and the HTTP client that will be used to make backend calls.
- Implementation of the different services: creation of each service following these rules:
 - one function = one HTTP call
 - always using encoded URLs from `endpoint.ts` instead of hardcoded ones
 - always including credentials to allow cookies
 - always giving meaningful names and specification to clearly understand what each service does
 - returning an `Observable` of the corresponding DTO when applicable.

Here is a full example respecting all these rules:

```
/**
 * @return the currently connected user
 */
public getConnectedUser(): Observable<UserResponseDto> {
  return this.httpClient.get<UserResponseDto>(
    `${this.baseUrl}${API_ENDPOINTS.USER.PROFILE}`,
    { withCredentials: true });
}
```

There are a lot of services that are available, don't hesitate to go into each folder to see what is already at your disposal.

5.3.2 features

The **features** folder is where all the user-facing screens of the application are stored. It holds the frontend feature modules, combining UI components, pages, routing and feature-scoped utilities. While **core/service** is responsible for all backend communication, **features** is responsible for presenting data to the user and wiring together core services, translations and local presentation logic.

```
features/
|- auth/
|   |- pages/
|   |- components/
|   |- auth.routes.ts
|   |- *.spec.ts
|- map/
|   |- pages/
|   |- components/
|   |- map-utils.ts
|   |- map.component.ts
|   |- *.spec.ts
|- ...
```

Each feature should ideally follow the same internal layout:

- **pages/** — top-level view components acting as screen containers. They handle data subscriptions, navigation and pass data down to presentational components.
- **components/** — smaller, reusable presentational pieces scoped to the feature. They receive data via `@Input()` and emit user events via `@Output()`.
- ***.routes.ts** — feature routing file exporting a route array, using lazy-loading where appropriate. These are wired into the central `app.routes.ts`.

- **Utilities** — small, pure helper functions kept inside the feature folder (e.g. `map-utils.ts`, `tile-utils.ts`). Prefer pure functions and keep them unit-tested.
- `*.spec.ts` — unit tests placed next to the files they test, covering pages, key components and utilities.

All the features respect the same set of rules regarding their relation to `core/service`:

- **No direct HTTP calls.** Features must never import or use `HttpClient` directly. All backend communication is handled exclusively by the services in `core/service`, which return `Observable<DTO>`.
- **Consume core services.** Pages and components inject core services and subscribe to their Observables to obtain data. The central endpoint definitions from `endpoint.ts` and the application routes from `app.routes.ts` are never duplicated inside a feature.
- **Single responsibility.** A feature component handles UI and local presentation logic only. Side effects, business rules and state mutations belong in services (core or feature-scoped), not in templates.

The naming and file conventions also follow a guideline we strove to adhere to across features:

- Components: `*.component.ts` + `*.component.html` + `*.component.css`, one concern per component.
- Routes: `*.routes.ts`, exporting the feature route array with lazy-loading. Example: `auth.routes.ts`.
- Tests: `*.spec.ts` placed next to the file under test.

There are several features already available; do not hesitate to explore each folder to see the patterns already in place before implementing a new one.

5.3.3 shared

The shared package is, as its name indicates, where all the components shared across our pages are located. The folder is organized as follows:

```
shared/  
  |- components/  
  |- models/
```

5.3.3.1 components

This folder contains all the shared Angular components. Whenever a component is reused across several pages and is sufficiently large, it should be placed here. Some shared components are already defined, so make sure that what you want to create does not already exist before adding a new one.

5.3.3.2 models

This folder contains all the DTOs (Data Transfer Objects) used to communicate with the backend. As mentioned in the [Data Formats Between Layers](#) section, DTOs define the structure that data must follow when interacting with the backend, ensuring that only relevant data is transferred and exposed. Therefore, this folder contains the TypeScript representations of these DTOs (they are very similar to the backend ones, except that they follow TypeScript conventions). The DTOs are organized into several folders based on business logic.

```
models/
  |- AdminModel/
  |- EvaluationFormModel/
  |- MapModel/
  |- MeasureModel/
  |- UserModel/
  |- MessageDto.ts
```

- AdminModel: All the DTOs related to the admin service
- EvaluationFormModel: All the DTOs related to the evaluation form service
- MapModel: All the DTOs related to the map service
- and so on.

5.3.4 Transloco

Transloco is the library we used for our project to handle translation. It is an internationalization library for Angular that allows us to define translations for our content in multiple languages and switch between them dynamically at runtime.

We use two configuration files in the application to set up Transloco :

```
app/
  |- .../
  |- transloco.config.ts/
  |- transloco.loader.ts
```

In the assets folder, we have all the translation files:

```
app/
  |- background/
  |- i18n/
  |   |- en.json/
  |   |- fr.json
  |- icons
```

These files contain the translations for the different elements displayed in our HTML pages.

6 Adding new code

Now that you have understood the source code, or at least the main ideas behind its structure, you may want to add a new feature. Here is the roadmap of the steps you should follow.

Warning: This is not a full explanation on how to produce Springboot or Angular code (as it would take be too long to explain), but rather a summary of all the steps you should consider when you want to add a new feature.

6.1 Before adding the code

1. **Look through the codebase to ensure that the feature you want to add does not already exist:** Before jumping into anything, make sure that what you want to do has not already been implemented. Otherwise, it will be a waste of time.
2. **Take the time to think about how you are going to add the feature before implementing it:** Once you are sure that your feature does not already exist, you should take some time to think about the best way to integrate it into the codebase. This means considering the different possibilities for implementing it, rereading the parts of the documentation related to your feature, etc. It may seem unnecessary or not very fun, but it will greatly help you during the next steps.

6.2 Adding code to the backend

If you want to add code to the backend, you should follow the steps below:

1. **Decide in which package you will add your feature:** The first thing to do is decide in which package you want to add your feature. You can either add it to an existing package or create a new one. When making this decision, always try to respect the project's organization rules. This means that all features related to the same object or business element should be stored in the same package. For example, all features related to users are in the user package, while all features related to maps are in the map package.
2. **Write the feature code:** Now that you have chosen your package, you can start writing your code. Here is a non-exhaustive list of points you should focus on while coding:
 - Does my code respect the backend architecture by separating controllers, services, and repositories, and by using DTOs to transfer information to the frontend?
 - Do my controllers respect REST principles?
 - Do my DTOs expose only the necessary information to the frontend?

- Is my business logic located in the services rather than in the controllers?
- Can Spring Boot provide tools that could help me with my tasks?
- Does my code respect Java coding conventions? (naming, package, Encapsulation,...)
- Have the new libraries I am using been added to the pom.xml file?
- Is my code readable for other developers?

Honestly, there are many more things that could be added here, but the main idea is to always try to respect the architectural principles of the codebase while remembering that your code will probably be used by other developers in the future.

3. **Specify your feature:** Your code has been produced, which is great, but do not forget to document it. The project follows Javadoc conventions for documentation. If you are unsure how to document something, do not hesitate to look at what has already been done.
4. **Test your feature:** Even though we have not talked much about it, testing your feature is very important. All tests are located in the test folder (backend/src/test). We highly recommend unit testing all the code you have produced. You can also perform integration testing if you think it is necessary. If you don't know how to test, here is a series of videos explaining all you need to know about springboot testing: [Spring Boot Testing](#).
5. **Launch a build:** Once all the steps are completed, you can run a Maven build using "mvn clean test", which will automatically launch a Checkstyle analysis and run all the test suites on your code. If the build succeeds, you are good to go. Otherwise, you need to correct the errors until the build succeeds.

6.3 Adding code to the frontend

If you want to add code to the frontend, you should follow the steps below:

1. **Creating the DTOs:** The first thing you need to check when adding a new feature to the frontend is whether all the DTOs required for the services you need are defined in the [models](#) folder. If this is not the case, the first step is to add them into the it.
2. **Creating the endpoints and services:** Once you have all your DTOs, you should ask yourself: "Do I have all the services needed to create my webpage?" If not (for example, because you created new services in the backend but have not yet added them to the frontend), you need to add all the required services in the [service](#) folder. Do not forget to avoid directly writing endpoints as strings in your services. Instead, use the [rest-api-management](#) folder to define your endpoints.

3. **Creating shared components:** Now that the services are ready, you can almost start working on your new page. However, before jumping into development, it is a good idea to think about the following question: "Are there elements that could be reused across multiple webpages?" If the answer is yes, then you should probably create these components in the [components](#) section so they can easily be reused in the future.
4. **Creating the webpage itself:** Now that you have created all the DTOs, services, and shared components, you can start working on your webpage. Here are some elements you should consider while developing it:
 - Have the new routes I created been added to the `app.route.ts` file?
 - Have I added the translations ([Transloco](#)) of my HTML text into the `en.json` or `fr.json` files located in `GeoHealth/frontend/src/assets/i18n/`?
 - Is my TypeScript code properly typed?
 - Is my design easy for the user to understand?

Again, there is no single rule that can always be applied to successfully create a new webpage. The main thing you should keep in mind is to try to maintain consistency with the rest of the frontend and to create pages that are both readable and maintainable.

5. **Testing your webpage:** Unfortunately, we have not created automatic tests for the frontend yet. Therefore, the only way to test your page is to manually navigate to it and verify that it behaves as expected. It is not perfect, but it is still better than doing no testing at all.

7 CI/CD

7.1 Pipeline overview

The GeoHealth CI/CD pipeline is built on GitHub Actions and is split into two parts: [Continuous Integration](#) and [Continuous Deployment](#). Every pull request to `main` automatically runs the [Continuous Integration](#) workflow, which is composed of a test suite and a code quality analysis. Once a pull request is merged into `main`, the [Continuous Deployment](#) workflow is triggered, which deploys the code into the server.

7.2 Continuous Integration

7.2.1 Automated Testing

- **Purpose** : The purpose of the automated testing workflow is to launch tests automatically on every pull request, ensuring that new code does not break existing functionalities. The tests must all pass for the pull request to be mergeable. Additionally, Checkstyle is integrated into the Maven build to enforce a shared code style across the project. Any style violation will fail the build, just like a failing test would.
- **Technology** : Maven, Java 17, Checkstyle
- **Outcome** : fails if tests fail OR if Checkstyle violations are found.

7.2.2 Code Quality Analysis

- **Purpose** : The purpose of this workflow is to automatically analyse the code quality on every pull request as sonar is a static code analyser that allows us to detect bugs, vulnerabilities, and code smells.
- **Technology** : SonarCloud, Maven, Java 17
- **Outcome** : The results are accessible on the SonarCloud dashboard. A failure does not block the merge, but serve as important comments for the developers.

7.3 Continuous Deployment

7.3.1 Overview

The deployment is triggered automatically when a pull request is merged into `main`. GitHub Actions builds both the frontend and the backend, transfers the compiled artifacts to the university VM, and restarts the services.

7.3.2 Deployment Pipeline

In this section we will explain what the **deploy.yml** does. The deployment happens in three stages. We first build the frontend and the backend, these builds are then sent to the deployment server, and finally the server restarts to run the new files. The idea is that the server does not host the source code but only the compiled files.

Trigger

This workflow is triggered when a pull request is merged into **main**. Simply opening or closing a pull request without merging does not trigger the deployment.

Build

Two things are built on the GitHub Actions:

- Angular compiled in production mode with `npm run build`
- Spring Boot packaged into a JAR with `mvnw package` (tests skipped since they already ran in CI)

Transfer

Three things are sent to the VM:

- Angular static files → `/opt/geohealth/frontend`
- Spring Boot JAR → `/opt/geohealth/backend`
- `docker-compose.yml` → `/opt/geohealth`

Restart

The server must restart with the new files, the following instructions are executed on the server:

- Docker Compose starts the database
- The old spring boot process is killed
- `index.html` is fixed (the Angular CSR file rename)
- New JAR is launched with the production profile

7.4 Deployment Server

7.4.1 Software installed

- Java 17 (`openjdk-17-jre`)
- R 4.5.3 with the `irr` package (for managing metrics computation)
- GDAL 3.12.2 "Chicoutimi" (for managing map transformation)
- Docker and Docker Compose

- Nginx
- Certbot (for SSL certificate)

On the production VM, three services run simultaneously:

- **Nginx** : Acts as the entry point for all incoming traffic. It serves the Angular frontend and forwards API calls to the Spring Boot backend.
- **Spring Boot JAR** : Runs the backend application internally on port 8080. It is not directly accessible, all requests go through Nginx first.
- **PostgreSQL (Docker)** : Runs the database internally on port 5432. It is only accessible by the Spring Boot backend.

Nginx is the only service exposed to the outside world, listening on port 443 (HTTPS). When a request arrives, Nginx decides where to send it based on the URL:

- Any request to `https://geohealth.unamurcs.be/api/*` is forwarded to the Spring Boot backend running on `localhost:8080`. The `/api` prefix is stripped before forwarding.
- Any other request is served the Angular static files from `/opt/geohealth/frontend/browser/`, letting Angular handle the routing on the client side.

7.4.2 VM File Structure

```
/opt/geohealth/  
|  |- frontend/  
|  |   |- browser/          <- compiled Angular files  
|  |   |- backend/  
|  |       |- webgis-*.jar    <- compiled Spring Boot JAR  
|  |   |- docker-compose.yml  
|  |   .env                  <- production secrets (not in the git repo)  
|  |   app.log               <- Spring Boot logs
```

7.4.3 Nginx configuration

The Nginx configuration is located at `/etc/nginx/sites-available/geohealth`.

The configuration defines two server blocks. One for HTTPS (port 443) and one for HTTP (port 80). Both with identical routing rules. each bloc has the following sections:

- **location** / Serves the Angular application. The `try_files` directive redirects all routes to `index.html`, letting Angular handle client-side routing. Without this, refreshing any page other than the root would return a 404.

- **location /api/** Proxies all API calls to the Spring Boot backend on `localhost:8080`, stripping the `/api` prefix before forwarding. Timeouts are set to 700 seconds to accommodate long-running operations such as loading large map files. Cookies are forwarded to preserve JWT authentication.
- **location /tile/** Proxies map tile requests directly to Spring Boot. Tiles are handled separately as they are binary image data used by the map rendering engine.

7.4.4 SSL certification

HTTPS is handled by Let's Encrypt via Certbot. The certificate was obtained with:

```
sudo certbot --nginx -d geohealth.unamurcs.be
```

Certbot automatically renews the certificate every 90 days via a scheduled task.

7.4.5 GitHub Required Secrets

The following secrets must be configured in GitHub:

HOST: The IP address of the university VM.

USER: The SSH username.

PASSWORD: The SSH password.

PORT: The custom SSH port number.

SONAR_TOKEN: The SonarCloud authentication token.

7.4.6 Useful Commands

1. Spring Boot

Check if Spring Boot is running:

```
ps aux | grep java
```

View Spring Boot logs:

```
cat /opt/geohealth/app.log
```

2. Database

Check database status:

```
cd /opt/geohealth
```

```
docker-compose ps
```

Delete the database:

```
docker-compose down -v
```

3. Nginx

Check Nginx status:

```
sudo systemctl status nginx
sudo nginx -t
sudo systemctl restart nginx
```

4. Manual Restart

Restart everything manually:

```
cd /opt/geohealth
export $(cat .env | xargs)
docker-compose --env-file .env up -d
sudo pkill -f webgis || true
sleep 2
cp frontend/browser/index.csr.html frontend/browser/index.html
nohup java -jar backend/*.jar --spring.profiles.active=prod > app.log 2>&1 &
```